

ESP8266 Application Note

Firmware Download Protocol



Version 1.2
Espressif Systems
Copyright © 2018

About This Guide

This document introduces a downloading protocol for ESP8266 firmware.

Release Notes

Date	Version	Release notes
2016.05	V1.0	Initial release.
2017.06	V1.1	Added operation codes 09, 0a, 0b in Table 2-2.
2018.11	V1.2	- Updated Section A.3. References - Fixed format issues

Documentation Change Notification

Espressif provides email notifications to keep customers updated on changes to technical documentation. Please subscribe at <https://www.espressif.com/en/subscribe>.

Certification

Download certificates for Espressif products from <https://www.espressif.com/en/certificates>.

Table of Contents

- 1. Overview1
 - 1.1. Hardware Preparations1
 - 1.1.1. Hardware Settings1
 - 1.1.2. Hardware Connection1
 - 1.2. Download Procedure2
- 2. Transmission Protocol3
 - 2.1. Packet Header3
 - 2.2. Packet Body4
- 3. Firmware Image Format5
- A. Appendix - Programming Examples6
 - A.1. Checksum6
 - A.2. Erase Flash6
 - A.3. References7



1. Overview

1.1. Hardware Preparations

When ESP8266 is in the UART downloading mode, users can download firmware from an external MCU to ESP8266.

1.1.1. Hardware Settings

The hardware settings are shown in Table 1-1.

Table 1-1. Hardware Settings

Item	Settings
UART download mode	GPIO0 and GPIO15: pulled down GPIO2: pulled high
Baud rate	Auto-bauds
Data bit	8
Stop bit	1
Parity bit	None
Flow control	Disabled

1.1.2. Hardware Connection

The hardware connection is shown in Figure 1-1.

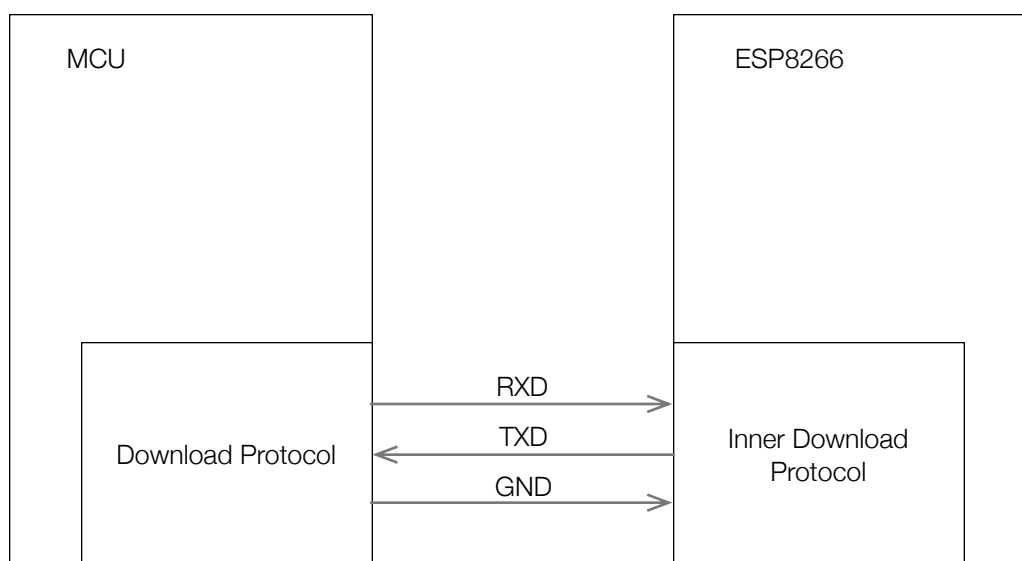


Figure 1-1. Hardware Connection



1.2. Download Procedure

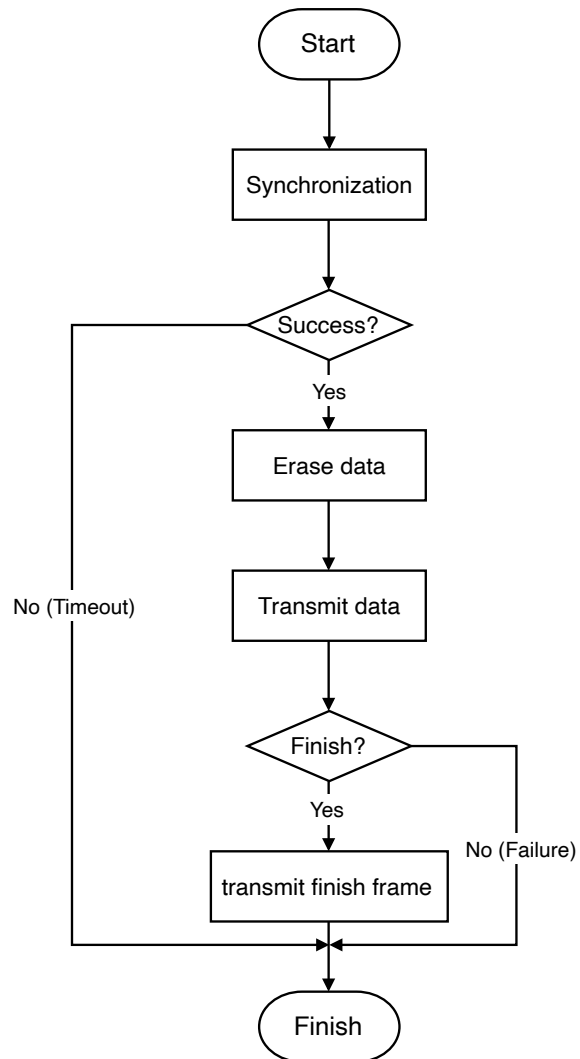


Figure 1-2. Download Procedure

- Synchronization: transmit sync frame to synchronize the baud rate.
- Erase data: erase the relevant flash sectors, according to the size and address of the firmware that will be downloaded into them.
- Transmit data: encapsulate the firmware into multiple frames and transmit them to ESP8266.
- Transmit finish frame: transmit the download-finish frame to ESP8266.



2. Transmission Protocol

The transmission protocol uses the Serial Line Internet Protocol ([SLIP](#)) framing.

- Each packet begins and ends with `0xC0`.
- All occurrences of `0xC0` and `0xDB` inside the packet are replaced with `0xDB 0xDC` and `0xDB 0xDD`, respectively.
- Inside the frame, the packet consists of a header and a variable-length body, as shown in Figure 2-1.
- All multi-byte fields are little-endian.

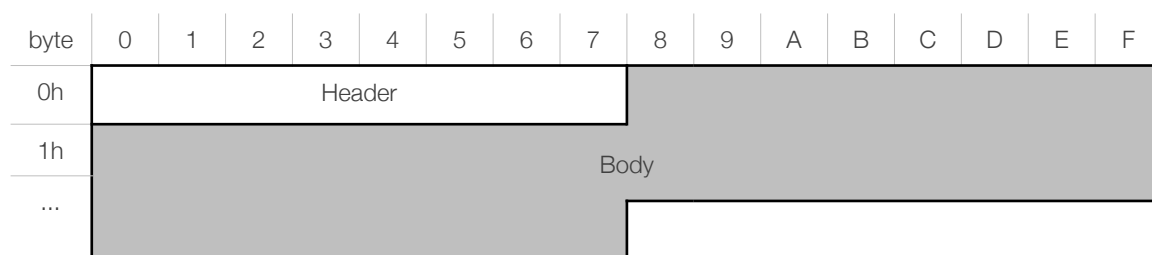


Figure 2-1. Packet Format

2.1. Packet Header

The format of the packet header is shown in Table 2-1.

Table 2-1. Packet Header Format

Byte	Data type	Request	Response
0	Type	Always <code>0x00</code> .	Always <code>0x01</code> .
1	Command	Operation code. Please refer to Table 2-2 for details.	
2~3	Data size	The size of packet body. Note: The data size is the same as the length of the packet body before <code>0xC0</code> and <code>0xDB</code> are replaced.	
4~7	Checksum/Response	XOR checksum of payload (the firmware data stored after the 16th byte of the packet body). For the checksum algorithm, please refer to Appendix - Programming Examples.	Response data.
8~n	Body	Depends on operation	
8	Status	-	Status flag, success (0) or failure (1).
9	Error	-	Success (null) or failure (error code).



Table 2-2. Operation Code

Codes	Name	Description
02	Flash Download Start	Erase the data in the flash. - Word0: the number of flash sectors being erased. Each sector is 4096 bytes. - Word1: the number of packet(s) being transmitted. - Word2: packet size, e.g., 0x400. - Word3: offset address. Note: For the sample codes of erasing data, please refer to Appendix - Programming Examples.
03	File Packet Send	Transmit data. - Word0: the size of data being written (filled with 0x400). - Word1: the sequence number of each transmitted packet. - Word2: 0x0 - Word3: 0x0
04	Flash Download Stop	Stop transmitting data.
08	Sync Frame Send	<code>sync_frame[36] = { 0x07, 0x07, 0x12, 0x20, 0x55 };</code>
09	Write register	Four 32-bit words: address, value, mask and delay (in microseconds).
0a	Read register	Address as 32-bit word.
0b	Configure SPI params	24 bytes of unidentified SPI parameters.

2.2. Packet Body

The packet body format is shown in Figure 2-2.

0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
Word0				Word1				Word2				Word3			
g...n															
Data															

Figure 2-2. Packet Body Format

The first 16 bytes (Word0 ~ Word3) are the description of the packet body, which is different when executing different commands.



3. Firmware Image Format

The firmware consists of a file header, and a number of data blocks (the size of blocks may be different), as shown in Figure 3-1. Multi-byte fields are little-endian.

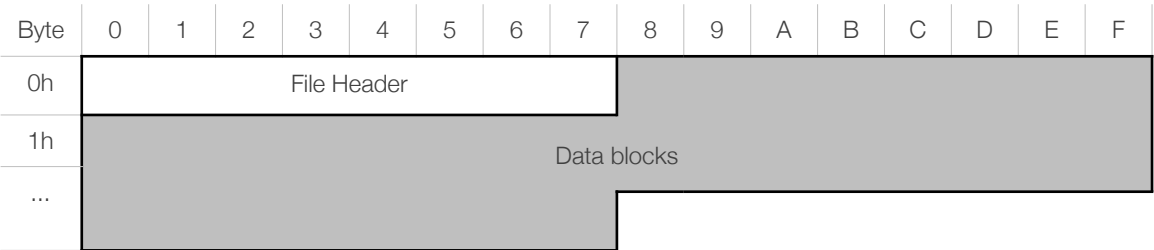


Figure 3-1. Firmware Image Format

The format of the file header is shown in Table 3-1.

Table 3-1. Firmware Format Description

Byte	Data type	Description
0	Magic Code	The value is always 0XE9.
1	Block Number	The number of blocks.
2	SPI Mode	The SPI working mode. 0x00: QIO mode 0x01: QOut mode 0x02: DIO mode 0x03: DOut mode
3	SPI Flash Info	SPI flash size and frequency. - High 4 bits: 0x0 = 512 kB; 0x1 = 256 kB; 0x2 = 1 MB; 0x3 = 2 MB; 0x4 = 4 MB - Low 4 bits: 0x0 = 40 MHz; 0x1 = 26 MHz; 0x2 = 20 MHz; 0xF = 80 MHz
4~7	Entry Address	CPU entry address.



A. Appendix - Programming Examples

A.1. Checksum

```
uint32_t espcomm_calc_checksum(unsigned char *data, uint16_t data_size)
{
    uint16_t cnt;
    uint32_t result;
    result = 0xEF;
    for(cnt = 0; cnt < data_size; cnt++)
    {
        result ^= data[cnt];
    }
    return result;
}
```

A.2. Erase Flash

```
#define BLOCKSIZE_FLASH 0x400
#define FLASH_DOWNLOAD_BEGIN 0x02
uint32 flash_packet[];
//uint32_t size:firmware real size,      uint32_t address: download offset address
int erase_flash(uint32_t size, uint32_t address)
{
    const int sector_size = 4096;
    const int sectors_per_block = 16;
    const int first_sector_index = address / sector_size;
    const int total_sector_count = ((size % sector_size) == 0) ?
                                    (size / sector_size) : (size / sector_size + 1);
    const int max_head_sector_count = sectors_per_block - (first_sector_index %
                                                            sectors_per_block);
    const int head_sector_count = (max_head_sector_count > total_sector_count) ?
                                    total_sector_count : max_head_sector_count;

    // SPIEraseArea function in the esp8266 ROM has a bug which causes extra area to be erased.
    // If the address range to be erased crosses the block boundary,
    // then extra head_sector_count sectors are erased.
    // If the address range doesn't cross the block boundary,
    // then extra total_sector_count sectors are erased.
```



```
const int adjusted_sector_count = (total_sector_count > 2 * head_sector_count) ?  
    (total_sector_count - head_sector_count):  
    (total_sector_count + 1) / 2;  
erase_size = adjusted_sector_count * sector_size;  
flash_packet[0] = erase_size;  
flash_packet[1] = (size + BLOCKSIZE_FLASH - 1) / BLOCKSIZE_FLASH;  
flash_packet[2] = BLOCKSIZE_FLASH;  
flash_packet[3] = address;  
espcmm_send_command(FLASH_DOWNLOAD_BEGIN, (unsigned char*) &flash_packet, 16);  
}
```

A.3. References

espressif/esptool: <https://github.com/espressif/esptool>



Espressif IoT Team
www.espressif.com

Disclaimer and Copyright Notice

Information in this document, including URL references, is subject to change without notice.

THIS DOCUMENT IS PROVIDED AS IS WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any proprietary rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

The Wi-Fi Alliance Member logo is a trademark of the Wi-Fi Alliance. The Bluetooth logo is a registered trademark of Bluetooth SIG.

All trade names, trademarks and registered trademarks mentioned in this document are property of their respective owners, and are hereby acknowledged.

Copyright © 2018 Espressif Inc. All rights reserved.